UNIVERSITAT POLITÈCNICA DE CATALUNYA
BARCELONATECH
Facultat d'Informàtica de Barcelona

Enunciat de la pràctica de laboratori

Lab 4:
Display gràfic de cristall líquid GLCD

L4. GLCD

L4(A) Funcions bàsiques GLCD + convertidor Analògic/Digital

L4(B) Rellotge amb timer

1 Objectius

L'objectiu d'aquesta pràctica és el desenvolupament de codi per presentar informació en una pantalla de cristall líquid o GLCD (Graphic Liquid Crystal Display). Els exercicis sobre la GLCD estan dividits en dues parts pràctiques. Cadascuna de les parts es farà en una sessió de laboratori diferent.

La primera part anomenada **L4(A),** és una introducció al funcionament del display gràfic i consta d'unes subrutines bàsiques per a programar la pantalla gràfica. També haureu de configurar un convertidor analògic-digital que mesuri el valor d'un potenciòmetre i pintar el valor mesurat per pantalla.

A la segona part de la pràctica anomenada **L4(B)**, farem servir els timers del microcontrolador per programar un rellotge que es presentarà a la pantalla gràfica.

2 Introducció

A la figura 1 podeu veure l'esquema de Proteus de la pantalla GLCD connectada al microcontrolador.

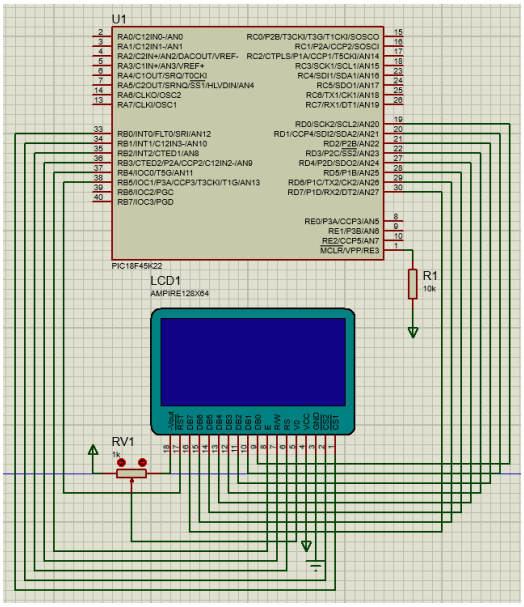

Figura 1. Connexió de la GLCD al PIC

Fixeu-vos que s'ha connectat la pantalla als ports B i D del microcontrolador. El port B té una sèrie de senyals de control de la pantalla (Enable, Reset, Read/Write...) i el port D serà un bus de 8 bits per on s'envien dades/informació a la pantalla (posició a pintar, caràcters a pintar, ordres d'esborrar, moure cursor, etc.). Podeu veure els detalls de les connexions de la GLCD a la taula 1, extreta del datasheet de la pantalla i disponible a Atenea.

Pin No.	Symbol	Level	Description
1	$\overline{CS1}$	L	Select Segment 1 ~ Segment 64
2	$\overline{CS2}$	L	Select Segment 65 ~ Segment128
3	Vss	0V	Ground
4	V_{DD}	5.0V	Supply voltage for logic
5	V_O	(Variable)	Operating voltage for LCD
6	D/I	H/L	H: Data , L: Instruction
7	R/W	H/L	H: Read(MPU Module) , L :Write(MPU→ Module)
8	E	H	Enable signal
9	DB0	H/L	Data bus line
10	DB1	H/L	Data bus line
11	DB2	H/L	Data bus line
12	DB3	H/L	Data bus line
13	DB4	H/L	Data bus line
14	DB5	H/L	Data bus line
15	DB6	H/L	Data bus line
16	DB7	H/L	Data bus line
17	RST	L	Reset the LCM
18	VEE		Negative Voltage Output
19	A		Power supply for B/L(+)
20	K		Power supply for B/L(-)

Taula 1. Pinout de la GLCD

Com a documents annexos a aquesta pràctica, trobareu:

- Projecte base de Proteus amb la pantalla GLCD (`L4_GLCD_Intro.pdsprj`).
- Datasheet de la GLCD (`WDG0151-TMI-V-N00-Specification.pdf`).

A la següent figura, es mostra l'aspecte del projecte base `L4_GLCD_Intro.pdsprj` carregat a Proteus:

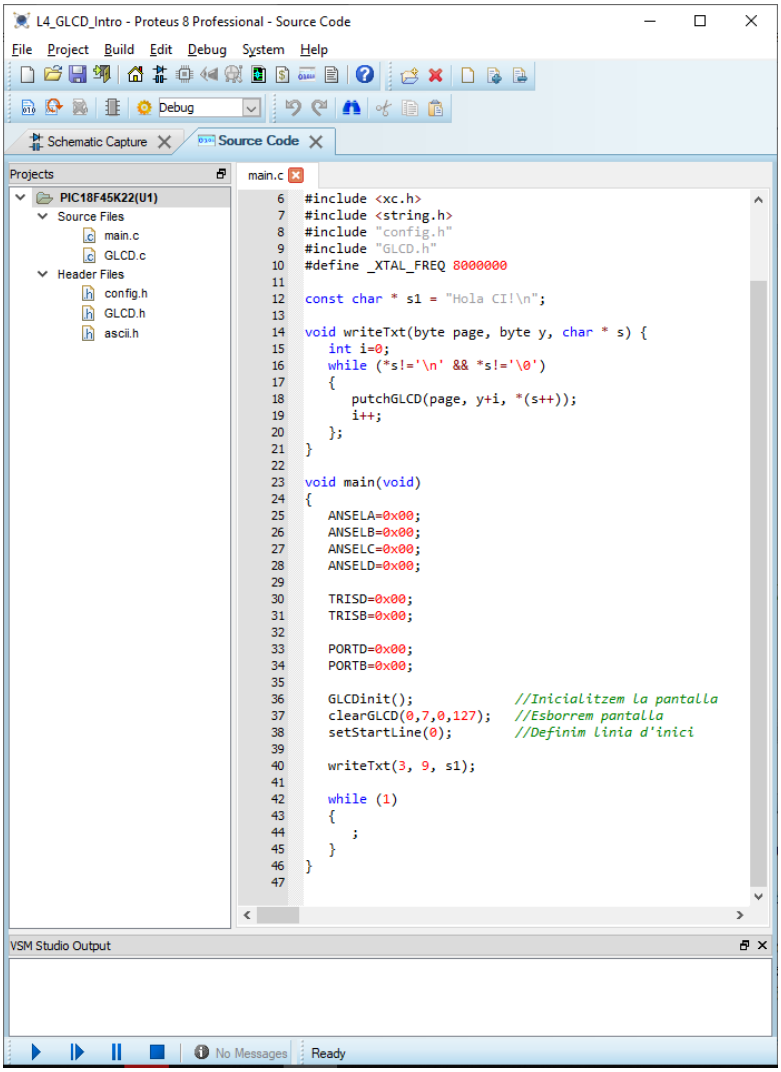

Figura 2. Captura del projecte a Proteus.

Com podeu observar, en el projecte hi han diferents fitxers `.c` i `.h` per poder usar llibreries, tenir fitxers de configuració, de dades, etc.
Concretament teniu el fitxer `main.c` amb el codi principal que configura i utilitza la pantalla, el fitxer `GLCD.c` amb les funcions de baix nivell de la pantalla, el fitxer `GLCD.h` amb les capçaleres de les funcions i el fitxer `ascii.h` on es defineixen els patrons per poder pintar caràcters a la pantalla.

Si executeu el programa, heu de veure la pantalla funcionant amb el següent text:

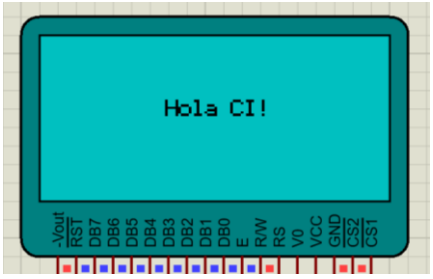

Figura 3. Demo de funcionament de l'exemple.

A partir d'aquest programa d'exemple, s'espera que desenvolupeu el vostre treball que s'especifica a les pàgines següents.

3 Treball Previ L4 (A)

Estudiar els diferents fitxers entregats amb la pràctica. Manuals i **sobretot el codi de la llibreria** `GLCD.h` **i** `GLCD.c` **per fer servir la pantalla**.

Modificar el projecte base perquè realitzi les següents funcions:

1. Fer una presentació que mostri un text per pantalla amb el nom de la pràctica "L4A GLCD" i el noms i cognoms dels integrants del grup al centre de la pantalla durant 1 segon.

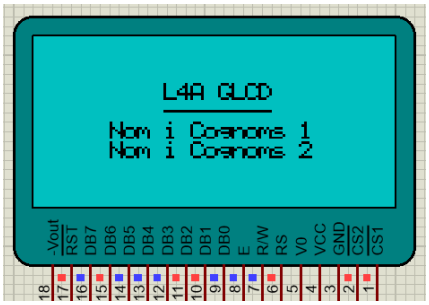

Figura 4. Pantalla de presentació.

2. Afegiu un potenciòmetre d'1kΩ al vostre circuit. Podeu situar el potenciòmetre a qualsevol pin que vulgueu i que estigui disponible. Podeu fer servir el component POT-HG de la llibreria ACTIVE de Proteus.

3. Afegiu tres texts a la pantalla. Aquests texts indicaran el valor llegit del potenciòmetre a través del convertidor analògic-digital de 10 bits (entre 0 i 1023 en el cas del PIC). Un altre text indicarà el valor en tensió (entre 0.0 i 5.0 volts). I per últim un altre text que indiqui el tant per cent del valor del potenciòmetre (entre 0 i 100%). Podeu disposar-los a qualsevol lloc de la pantalla que vulgueu, excepte a la part inferior de la GLCD.

3. Dibuixeu una barra de progrés a la part inferior de la GLCD.

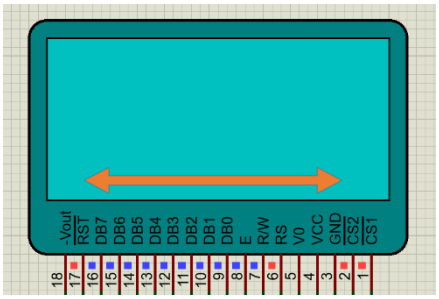

Figura 5. Ubicació on es situarà la barra de progrés.

5. La barra de progrés s'omplirà segons el valor llegit amb el convertidor analògic-digital connectat al potenciòmetre, de tal manera que si valor mesurat és 0, la barra estarà buida. Si es mesura 512, llavors la barra estarà mig omplerta. En el cas que es mesuri 1023, llavors la barra estarà plena. Entre aquests valors hi haurà tots els valors que convinguin per donar sensació de continuïtat a la barra de progrés.

Tingueu en compte que la pantalla té un tamany de 128x64 pixels. **A més el píxel 0,0 està a dalt a l'esquerra de la GLCD!**

4 Rúbrica treball Previ L4 (A)

	Iniciat (0-2.5 punts)	En desenvolupament (2.5-5.0 punts)	Aconseguit (5.0-7.5 punts)	Exemplar (7.5-10 punts)
Pantalla noms (1punt):	No hi ha	No es pot reaprofitar el codi	No està centrada	Funciona perfectament, és modular i es pot reaprofitar
Convertidor analògic/digital (3 punts):	No funciona bé o no compleix els requeriments temporals del PIC	Mal configurat	Ús dels recursos inapropiat, tipus de dades, conversions, escalats, etc.	Funciona perfectament, les operacions fan bon ús dels recursos, el tipus i conversió de dades és correcte, el temps de conversió està ben seleccionat
Texts (2 punts):	No hi ha text	El text no funciona bé, ja que deixa valors de dades anteriors	Funciona bé però es refresca massa ràpidament i la sensació és poc agradable	Funciona perfectament, i les conversions de dades estan ben justificades
Barra de progrés (4 punts):	No funciona	Funciona malament, hi han problemes amb la barra, els seus límits i el refresc de la pantalla	Funciona bé però és massa simple, o no té una mida fixa, o es refresca massa ràpidament i la sensació és poc agradable	Funciona perfectament, l'estructura del codi és bona, s'aprofiten correctament els recursos del microcontrolador, es fan servir funcions que siguin reutilitzables

5 Treball Previ L4 (B)

L'objectiu d'aquesta segona part de la pràctica és familiaritzar-se amb l'ús dels *Interval Timers*, i comprendre com podem gestionar els diferents esdeveniments generats al microcontrolador amb la utilització del tractament d'interrupcions.

Habitualment, en la programació de microcontroladors, necessitem comptar el temps transcorregut per tal d'executar accions periòdiques als nostres programes. Els microcontroladors ens ofereixen uns perifèrics que s'encarreguen de comptar temps de manera autònoma: els ***Timers***.

Utilitzareu el Timer0 per tal de comptar el temps transcorregut, i quan hagi passat 1 dècima de segon, actualitzareu el valor del temporitzador que mostreu per la GLCD.

Per a saber quan ha transcorregut 1 dècima de segon, podríem consultar contínuament el valor del comptador de Timer0, però seria una estratègia bastant ineficient que consumiria massa % de CPU del PIC. Per evitar aquest problema, farem servir les interrupcions generades pel perifèric Timer.

Tasques a realitzar

El programa a realitzar consisteix en un temporitzador digital amb el format hh:mm:ss.d indicant hores, minuts, segons i dècimes. El comptador de temps es podrà posar en marxa, aturar i resetejar mitjançant el polsador del pin RE0 de la EasyPIC. També afegirem una barra de progrés que anirà omplint-se a mesura que passa el temps, podeu recuperar la barra de la pràctica L4(A).

Tingueu present que la simulació temporal que fa Proteus té certes limitacions. És probable que el temps comptat pel temporitzador a la simulació presenti petites desviacions respecte a la realitat.

El detall de les tasques a realitzar és el següent:

1. Tornar a presentar la pantalla inicial que mostri el noms i cognoms dels integrants del grup i el text "L4B GLCD".

2. **Programeu una subrutina** `tic()` que s'executi cada dècima de segon mitjançant interrupcions periòdiques d'alta prioritat provinents del Timer0. Aquesta rutina és l'encarregada de cridar les accions a realitzar cada 0.1 segons, des de la rutina d'atenció a la interrupció del timer 0.

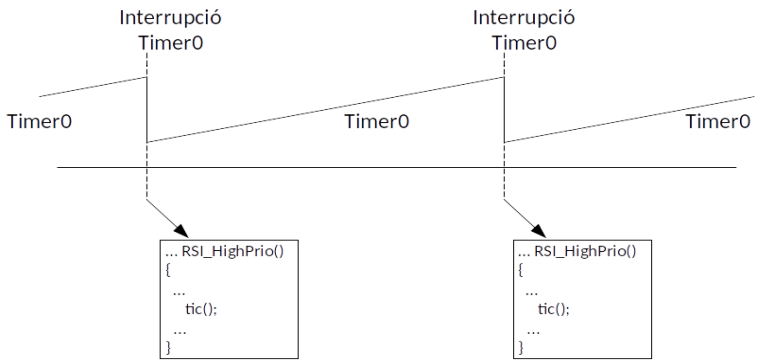

Figura 6. Esquema de funcionament del comptador del timer, la generació d'interrupcions i la crida de la funció `tic()` a la rutina d'atenció de la interrupció.

3. Afegiu un text que mostri el temps transcorregut amb el format (hh:mm:ss.d), podeu ubicar-lo a qualsevol punt de la pantalla GLCD que considereu adient. Programeu un procediment `updateGLCD(·)` que actualitzi el seu valor.

4. Actualitzeu la barra de progrés de tal manera que a l'instant de temps t=0 s, la barra de progrés estigui buida. Al temps t=5s, la barra estigui al 50%. I quan el temps sigui t=10s, la barra estigui al 100%. Entre aquests valors hi haurà tots els valors que convinguin per donar sensació de continuïtat a la barra de progrés.

5. El temporitzador no sempre estarà en marxa. Podrà estar en tres estats: *Ready*, *Running* o *Stopped*. El canvi entre estats el fareu amb el **flanc ascendent de la pulsació del botó RE0**. Gestionareu les activacions del polsador RE0 mitjançant la tècnica de polling o d'enquesta. Implementeu una estratègia anti-rebots que eviti la detecció de flancs no desitjats degut a problemes mecànics del botó.

6. El funcionament del temporitzador serà el següent. Al iniciar el programa, el sistema estarà en estat READY i apareixerà el text "Ready" en algun punt de la pantalla que considereu adient. El temporitzador s'inicialitzarà a 00:00:00.0. La barra de progrés estarà buida.

Una pulsació del botó RE0 farà que passi a RUNNING i comenci a comptar. A la pantalla GLCD mostrareu el text "Running...". També s'haurà de mostrar el valor actualitzat cada 0.1 segons del temporitzador i actualitzar la barra de progrés quan faci falta.

Una nova pulsació del botó RE0 farà que el programa passi automàticament a STOPPED. S'ha d'indicar amb el text "Stopped!". La barra de progrés deixarà d'omplir-se en aquests estat i mantindrà el valor actual.

La següent pulsació del botó RE0 netejarà els comptadors al valor inicial, i passarem de nou a l'estat READY. S'ha de mostrar per pantalla el text "Ready" com a l'estat inicial. També es buidarà la barra de progrés.

Hi ha una última condició a tenir en compte, que succeirà quan el temporitzador arribi a 10 segons, 00:00:10.0. En aquest instant la barra quedarà totalment plena i l'estat serà també STOPPED, havent de mostrar també el text "Stopped!". També es podrà sortir d'aquest estat al apretar el botó RE0 i anant cap l'estat READY com s'explica en el paràgraf anterior.

```c
// a proposal for main.c ..... just to inspire you
#include <xc.h>
#include "config.h"
#include "GLCD.h"
....

#define _XTAL_FREQ 8000000 // needed for __delay_ms function

typedef enum{
   Ready=0,
   Running,
   Stopped
}state_t;

// global Variables
....

void tic(void)
{
    //Coses a fer periòdicament
}

// interrupt service routine for handling high priority Timer0
void interrupt rutinaAtencioInterrupcio (void)
...
    if (timer0Flag) {
       tic();
    }


// falling edge detection and debouncing
??? inputDetector(·) {
....
}

// initialize PORTs, timer0 and basic PIC resources
void configPIC() {
....
}

void main(void) {
....
  configPIC();

  ....


  while (1) { // MAIN LOOP starts here
    if ( inputDetector(·) ) { // check raising edge
      switch(state) {
        // depending on the current state, handle the transition
        ....
      }
    }

    updateGLCD(·); // show things on the GLCD smartly!
    ....
  }// MAIN LOOP ends here
}
```

6 Rúbrica treball Previ L4 (B)

	Iniciat (0-2.5 punts)	En desenvolupament (2.5-5.0 punts)	Aconseguit (5.0-7.5 punts)	Exemplar (7.5-10 punts)
Pantalla noms (0.5 punts):	No hi ha	No es pot reaprofitar el codi	No està centrada	Funciona perfectament
Transició botó RE0 (flanc ascendent) (1 punt):	Hw mal dissenyat i funcionament erroni	Hw ben dissenyat però funcionament erroni	Funcionament correcte però no hi ha detecció de flancs	Funcionament correcte amb detecció de flancs reutilitzable
Transició botó RE0 (anti-rebots) (1.5 punts):	No hi ha detecció d'anti-rebots	L'estratègia anti-rebots és molt bàsica	L'estratègia anti-rebots funciona correctament però el codi no és modular ni reutilitzable	L'estratègia anti-rebots funciona perfectament, el codi és modular i pot configurar-se fàcilment per diferents botons, el codi es pot reutilitzar i consta del seu arxiu *.h i *.c com si es tractés d'una llibreria
Programació Timer0 (2.5 punts)	Hi han més de 3 errors en la programació dels registres del timer i interrupcions	Hi han entre 2 i 3 errors en la programació dels registres del timer i interrupcions	Hi han entre 1 i 2 errors en la programació dels registres del timer i interrupcions	No hi han errors en la programació dels registres del timer i interrupcions
Màquina d'estats (1.5 punt):	No hi ha estats ni codi per fer les transicions	No hi ha estats o no hi ha codi per fer les transicions	O els estats o les transicions tenen algun error	Estats i transicions perfectament implementats fent servir eines de programació que permetin descriure els estats de forma eficient a l'hora d'analitzar el codi
Pantalla GLCD (3 punts)	La pantalla no presenta la informació correctament, apareixen punts on no toca, s'actualitza la pantalla en llocs del codi no adients	La pantalla no presenta la informació correctament, o apareixen punts on no toca, o s'actualitza la pantalla en llocs del codi no adients	La pantalla funciona bé però es fan més actualitzacions de les necessàries o l'estructura del codi no està ben raonada	Funciona perfectament, només s'actualitzen els elements que són necessaris, s'aprofiten bé tots els recursos i l'estructura del codi és correcta